МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

Отчет

по производственной практике

ПП.02.01 Технология разработки программного обеспечения
(индекс по РУП и наименование производственной практики)

Профессионального модуля ПМ.02 Осуществление интеграции программных модулей

Специальность 09.02.07 «Информационные системы и программирование»

Обучающийся _____ Катаев Лука Евгеньевич
(подпись) (фамилия, имя, отчество)

Группы ИПо-41

Руководитель практической подготовки от профильной организации

Индивидуальный предприниматель
(должность)

_____ Бокова Эдита Алексеевна
(подпись) (фамилия, имя, отчество)

МП «16» марта 2026 года

Руководитель практической подготовки от института

_____ Берестов Дмитрий Борисович
(подпись) (фамилия, имя, отчество)

«16» марта 2026 года

Пермь, 2026 год

Оглавление

ВВЕДЕНИЕ 3
РАЗДЕЛ 1. ОРГАНИЗАЦИОННО-ПОДГОТОВИТЕЛЬНЫЙ ЭТАП ПРОХОЖДЕНИЯ ПРАКТИКИ НА ПРЕДПРИЯТИИ 5
Тема 1.1. Инструктаж по прохождению производственной практики и правилам безопасности работы на предприятии 5
РАЗДЕЛ 2. ОЗНАКОМЛЕНИЕ СО СТРУКТУРОЙ И ХАРАКТЕРОМ ДЕЯТЕЛЬНОСТИ ПОДРАЗДЕЛЕНИЯ 7
Тема 2.1. Ознакомление с организацией работы на предприятии или в структурном подразделении 7
Тема 2.2. Ознакомление с должностными и функциональными обязанностями 8
РАЗДЕЛ 3. РАБОТА НА РАБОЧИХ МЕСТАХ ИЛИ В ПОДРАЗДЕЛЕНИЯХ ПРЕДПРИЯТИЯ 9
Тема 3.1. Выполнение производственных заданий 9
3.1.1. использование выбранной системы контроля версий; 9
3.1.2. разработка программного обеспечения 10
3.1.3. Использование методов для получения кода с заданной функциональностью и степенью качества 31
3.1.4. Инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования; 33
3.1.5. Выполнение отладки программного модуля с использованием специализированных программных средств; 35
3.1.6. Осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения 37
3.1.7. Интегрирование программных модулей 40
3.1.8. Верификация и аттестация программного обеспечения 42
ЗАКЛЮЧЕНИЕ 44
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ 46

ВВЕДЕНИЕ

Практика имеет целью комплексное освоение студентами всех видов профессиональной деятельности по специальности 09.02.07 «Информационные системы и программирование», формирование общих и профессиональных компетенций, а также приобретение необходимых умений и опыта практической работы по специальности.

Производственная практика направлена на освоение обучающимися профессиональных компетенций в процессе самостоятельного выполнения определенных видов работ, необходимых для осуществления профессиональной деятельности в максимально приближенных к производству условиях, в рамках модулей ППССЗ по основным видам профессиональной деятельности для освоения методов и приемов практического применения прикладных программных продуктов для программного обеспечения компьютерных систем.

В настоящей работе представлен отчет о прохождении производственной практики студента-практиканта. Основными видами деятельности которой являлись: разработка требования к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент; моделирование процесса разработки программного обеспечения

Задачи практики (освоение обучающимися профессиональных и общих компетенций):

1. Выполнять интеграцию модулей в программное обеспечение;
2. Выполнять отладку программного модуля с использованием специализированных программных средств;
3. Осуществлять разработку тестовых наборов и тестовых сценариев для программного обеспечения;
4. Производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования;
5. Выбирать способы решения задач профессиональной деятельности, применительно к различным контекстам;

6. Осуществлять поиск, анализ и интерпретацию информации, необходимой для выполнения задач профессиональной деятельности

7. Планировать и реализовывать собственное профессиональное и личностное развитие;

8. Работать в коллективе и команде, эффективно взаимодействовать с коллегами, руководством, клиентами

9. Осуществлять устную и письменную коммуникацию на государственном языке Российской Федерации с учетом особенностей социального и культурного контекста.

10. Использовать информационные технологии в профессиональной деятельности

11. Пользоваться профессиональной документацией на государственном и иностранном языках

РАЗДЕЛ 1. ОРГАНИЗАЦИОННО-ПОДГОТОВИТЕЛЬНЫЙ ЭТАП ПРОХОЖДЕНИЯ ПРАКТИКИ НА ПРЕДПРИЯТИИ

Тема 1.1. Инструктаж по прохождению производственной практики и правилам безопасности работы на предприятии

Общие функциональные обязанности:

Сотрудник обязан добросовестно выполнять должностные инструкции, соблюдать трудовую дисциплину и правила внутреннего распорядка. В обязанности входит бережное отношение к имуществу предприятия, соблюдение сроков выполнения работ и требований к качеству. Сотрудник должен своевременно проходить инструктажи по технике безопасности и соблюдать конфиденциальность служебной информации.

Правила техники безопасности на предприятии:

Запрещается курение в неотведенных местах, использование открытого огня и блокирование аварийных выходов. Все сотрудники должны знать план эвакуации и расположение первичных средств пожаротушения. При обнаружении возгорания необходимо немедленно сообщить руководству и приступить к тушению с помощью огнетушителя. Проходы и эвакуационные пути должны содержаться свободными.

Техника безопасности на рабочем месте программиста:

Рабочее место должно содержаться в чистоте, системный блок располагаться на устойчивой поверхности. Запрещается размещение на технике посторонних предметов и жидкостей. Экран монитора должен находиться на расстоянии 50-70 см от глаз, высота кресла регулироваться для обеспечения правильной осанки. Необходимо делать перерывы каждый час для снижения зрительного напряжения.

Правила работы с электрическими приборами:

Перед включением оборудования необходимо убедиться в целостности кабелей питания и вилок. Запрещается самостоятельный ремонт energized equipment и использование неисправных электроприборов. При эксплуатации источников бесперебойного питания следует соблюдать нагрузочные характеристики. Все розетки должны быть заземлены, запрещается использование переходников и удлинителей без предохранителей. При прекращении работы все электроприборы должны быть отключены от сети.

РАЗДЕЛ 2. ОЗНАКОМЛЕНИЕ СО СТРУКТУРОЙ И ХАРАКТЕРОМ ДЕЯТЕЛЬНОСТИ ПОДРАЗДЕЛЕНИЯ

Тема 2.1. Ознакомление с организацией работы на предприятии или в структурном подразделении

1. Режим работы предприятия

Салон красоты работает ежедневно с 9:00 до 21:00, без выходных. Для мастеров установлен гибкий график сменности: утренняя (9:00-15:00) и вечерняя (15:00-21:00). Администраторы работают по скользящему графику 2/2.

2. Форма организации труда и правила внутреннего распорядка

Применяется индивидуальная форма организации труда с элементами коллективной ответственности.

Ключевые правила:

- Обязательная предварительная подготовка рабочего места до начала приема клиентов
- Соблюдение профессиональной этики и дресс-кода
- Ежедневная сдача выручки и отчетность по оказанным услугам
- Запрет на использование личных мобильных устройств во время обслуживания клиентов

3. Структурные подразделения и штатное расписание

Организационная структура:

- Руководство (ИП)
- Административный отдел (2 администратора)
- Отдел косметологии (3 косметолога)
- Парикмахерский отдел (4 мастера)
- Отдел маникюра и педикюра (2 мастера)

Организация работы построена на принципах эффективного использования рабочего времени, четкого распределения обязанностей и персональной ответственности каждого сотрудника за результаты своего труда.

Тема 2.2. Ознакомление с должностными и функциональными обязанностями

Сотрудник занимает должность разработчика программного обеспечения и подчиняется непосредственно индивидуальному предпринимателю. Прием на работу и увольнение осуществляется приказом ИП. В своей деятельности сотрудник руководствуется требованиями технических заданий, действующим законодательством Российской Федерации и настоящей должностной инструкцией.

Основными должностными обязанностями являются разработка и сопровождение программного обеспечения для автоматизации бизнес-процессов салона красоты. Сотрудник осуществляет создание и поддержку баз данных, проводит тестирование и отладку программных модулей. В обязанности входит разработка технической документации, а также обучение сотрудников салона работе с внедряемым программным обеспечением.

Сотрудник имеет право вносить предложения по совершенствованию программного обеспечения и оптимизации бизнес-процессов. Он может запрашивать необходимую информацию для выполнения должностных обязанностей у других сотрудников салона. Разработчик вправе требовать соблюдения правил работы с программным обеспечением и участвовать в обсуждении проектов автоматизации.

Сотрудник несет ответственность за качество и своевременность выполнения поставленных задач. Он отвечает за сохранность конфиденциальной информации и соблюдение установленных сроков разработки. Дополнительно сотрудник обеспечивает работоспособность внедренного программного обеспечения и своевременное устранение возникающих сбоев.

РАЗДЕЛ 3. РАБОТА НА РАБОЧИХ МЕСТАХ ИЛИ В ПОДРАЗДЕЛЕНИЯХ ПРЕДПРИЯТИЯ

Тема 3.1. Выполнение производственных заданий

3.1.1. использование выбранной системы контроля версий;

В ходе выполнения производственных заданий для обеспечения надёжного хранения, версионного контроля и совместной работы с исходным кодом была выбрана система контроля версий Git совместно с облачным хостингом GitHub.

Обоснование выбора Git:

- Git является самой популярной распределённой системой контроля версий в мире (по данным Stack Overflow и GitHub Octoverse 2024–2025).
- Полностью бесплатен для учебных и личных проектов.
- Имеет отличную интеграцию с Visual Studio 2022 (встроенный Git-менеджер).
- Позволяет работать с ветками, быстро откатывать изменения и вести полную историю разработки.

Процесс организации контроля версий:

1. На платформе GitHub был создан репозиторий:
2. Репозиторий был клонирован в Visual Studio 2022 с помощью встроенного инструмента «Клонировать репозиторий».

3.1.2. разработка программного обеспечения

В рамках производственной практики была разработана подсистема «Учёт партнёров и расчёт индивидуальных скидок» для производственной компании «Мастер пол».

Выбор модели разработки

Из предложенных моделей разработки (каскадная, V-образная, прототипирования, RAD, итерационная, спиральная) была выбрана итерационная модель разработки.

Обоснование выбора:

- Требования заказчика были чётко сформулированы.
- Итерационная модель позволила разбить работу на логические итерации: создание базы данных и моделей, реализация бизнес-логики (расчёт скидки), разработка CRUD-операций, создание пользовательского интерфейса WPF, тестирование и отладка.
- Каждая итерация заканчивалась работающим прототипом, который сразу демонстрировался и корректировался.
- Модель позволила оперативно вносить изменения (например, добавление окна управления продукцией) без переписывания всего проекта.
- Каскадная модель была бы слишком жёсткой, а спиральная — избыточной для учебного проекта среднего объёма.

Инструменты, использованные для создания продукта

- Система контроля версий: Git + GitHub.
- СУБД: PostgreSQL 16.
- ORM: Entity Framework Core 8.
- Язык программирования: C#.
- Технология интерфейса: Windows Presentation Foundation (WPF).
- Дополнительно: Npgsql (драйвер PostgreSQL), XAML для разметки интерфейса.

Среда программирования

Основной средой разработки являлась Microsoft Visual Studio 2022 Community Edition. Выбор обоснован:

- Полная поддержка WPF и Entity Framework Core.
- Встроенные инструменты для работы с Git.
- Удобный дизайнер XAML (WPF Designer).
- Интеграция с PostgreSQL через NuGet-пакеты.
- Возможность отладки, профилирования и написания unit-тестов в одной среде.

Этапы разработки программного модуля

Разработка велась поэтапно в соответствии с итерационной моделью:

1. Выбор шаблона проекта — создан проект WPF (.NET Framework) + отдельная библиотека классов KataevLIB (.NET Framework).
2. Составление схемы информационной базы — спроектирована база PostgreSQL в 3НФ, созданы таблицы, схема app, роль app.
3. Документирование системы — составлены ER-диаграмма, диаграмма классов.
4. Разработка моделей и контекста (ApplicationContext, сущности Partner/Product/SalesHistory).
5. Реализация бизнес-логики — класс MainApp с методом расчёта скидки по заданным порогам.
6. Разработка форм — главная форма (список партнёров + детали + история продаж), окно управления продукцией, формы добавления/редактирования.
7. Тестирование — модульные тесты для расчёта скидки, ручное тестирование CRUD-операций.
8. Сопровождение — рефакторинг кода, обработка исключений, единый стиль интерфейса.

Создание роли, базы и настройка прав доступа

На начальном этапе требуется обеспечить наличие установленной системы управления базами данных (СУБД) PostgreSQL. В случае отсутствия программного обеспечения его инсталляция должна быть выполнена посредством загрузки дистрибутива с официального ресурса разработчика.

Следующим этапом является создание роли пользователя, которая будет использоваться для аутентификации и последующего администрирования базы данных. Для этого необходимо запустить интерактивную оболочку psql и выполнить команду создания роли с атрибутом входа (LOGIN). Синтаксис команды предполагает использование оператора CREATE ROLE.

После успешного создания объекта безопасности следует установить пароль для новой роли, воспользовавшись мета-командой \password, после чего ввести и подтвердить заданную комбинацию символов.

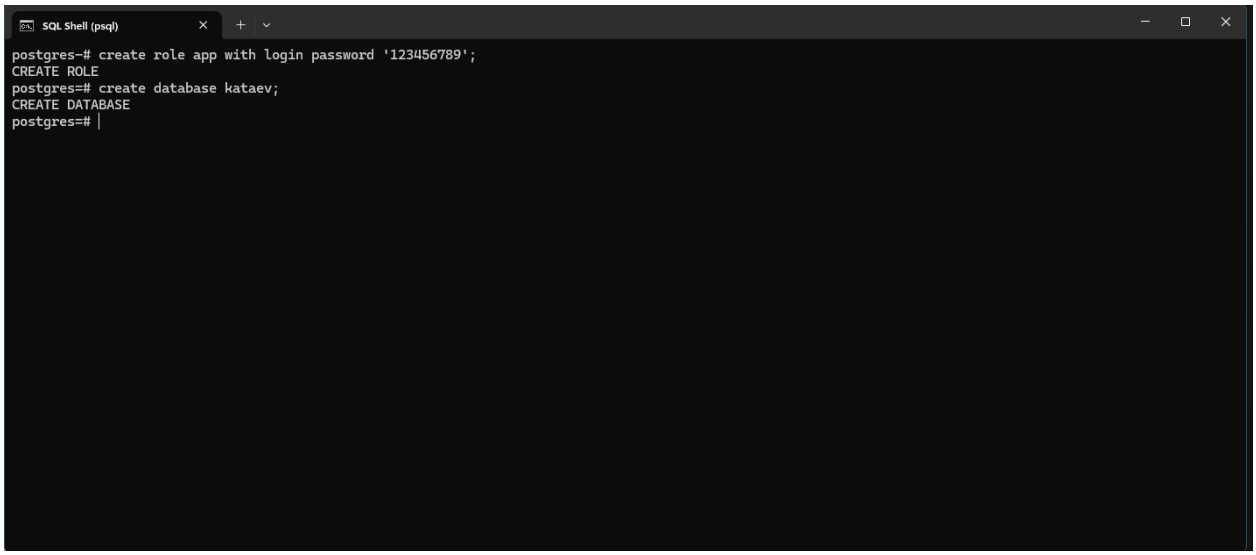

Рисунок 1 создание роли

По завершении настройки роли осуществляется создание базы данных с помощью SQL-команды CREATE DATABASE. После инициализации базы данных необходимо выполнить подключение к ней под учетной записью суперпользователя postgres.

Ключевым шагом конфигурации является делегирование необходимых привилегий ранее созданной роли. С этой целью применяется команда ALTER DEFAULT PRIVILEGES IN SCHEMA, позволяющая настроить права доступа по умолчанию для указанной схемы (в данном примере — схема app).

Конфигурация параметров данной команды обеспечивает возможность выполнения полного цикла операций с объектами базы данных: создания, модификации и удаления таблиц, работы со схемами, а также разработки и исполнения хранимых процедур и функций.

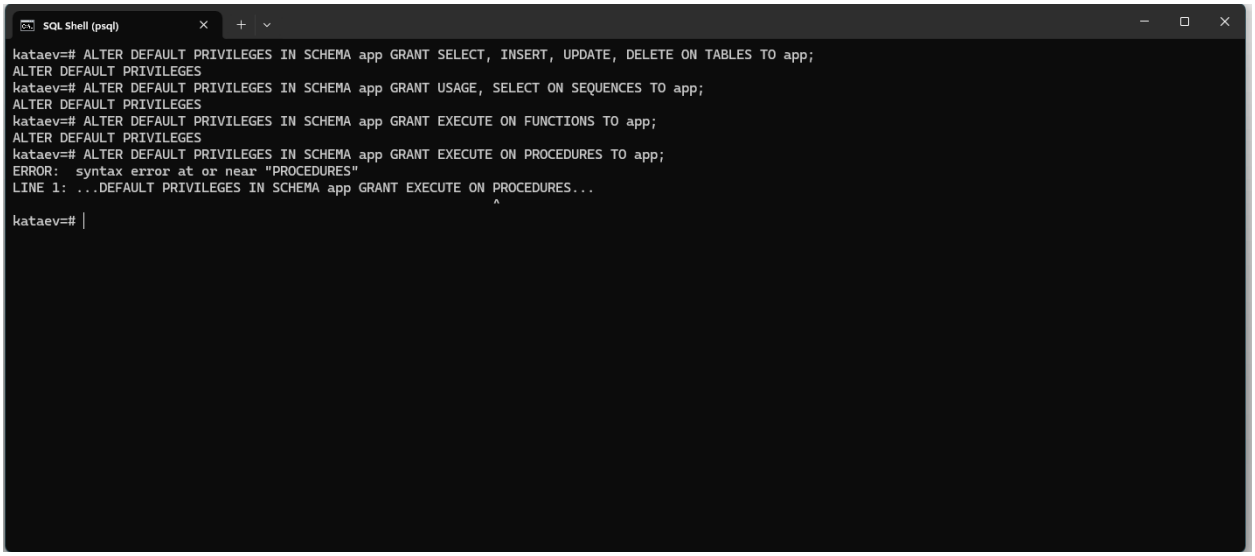

Рисунок 2 выдача прав

После применения настроек прав доступа система готова к эксплуатации: подключение к базе данных может быть осуществлено от имени настроенной пользовательской роли с полным набором разрешенных привилегий.

Создание таблиц базы данных

Структура базы данных была спроектирована в соответствии с требованиями предметной области и реализована средствами СУБД PostgreSQL 16.

Перед созданием таблиц была выполнена подготовка окружения: создана роль app с правом входа и паролем, создана база данных kataev, в которой была развёрнута схема app с назначением роли app в качестве владельца. Роли были предоставлены права на выполнение операций SELECT, INSERT, UPDATE, DELETE над таблицами схемы.

База данных содержит три таблицы, связанные отношением один-ко-многим.

Таблица KataevPartners хранит сведения о партнёрах компании: наименование, тип организации, рейтинг, адрес, ИНН, ФИО директора, контактный телефон и электронную почту. Первичным ключом является автоинкрементное поле KataevId.

Таблица KataevProducts содержит справочник продукции: наименование, артикул и тип. Первичным ключом также является KataevId.

Таблица KataevSalesHistory реализует связь между партнёрами и продукцией и хранит историю продаж: количество реализованной продукции и дату продажи. Поля KataevPartnerId и KataevProductId являются внешними ключами, ссылающимися на соответствующие таблицы с каскадным удалением.

Для ускорения выборок по внешним ключам и дате были созданы индексы на полях KataevPartnerId, KataevProductId и KataevSaleDate.

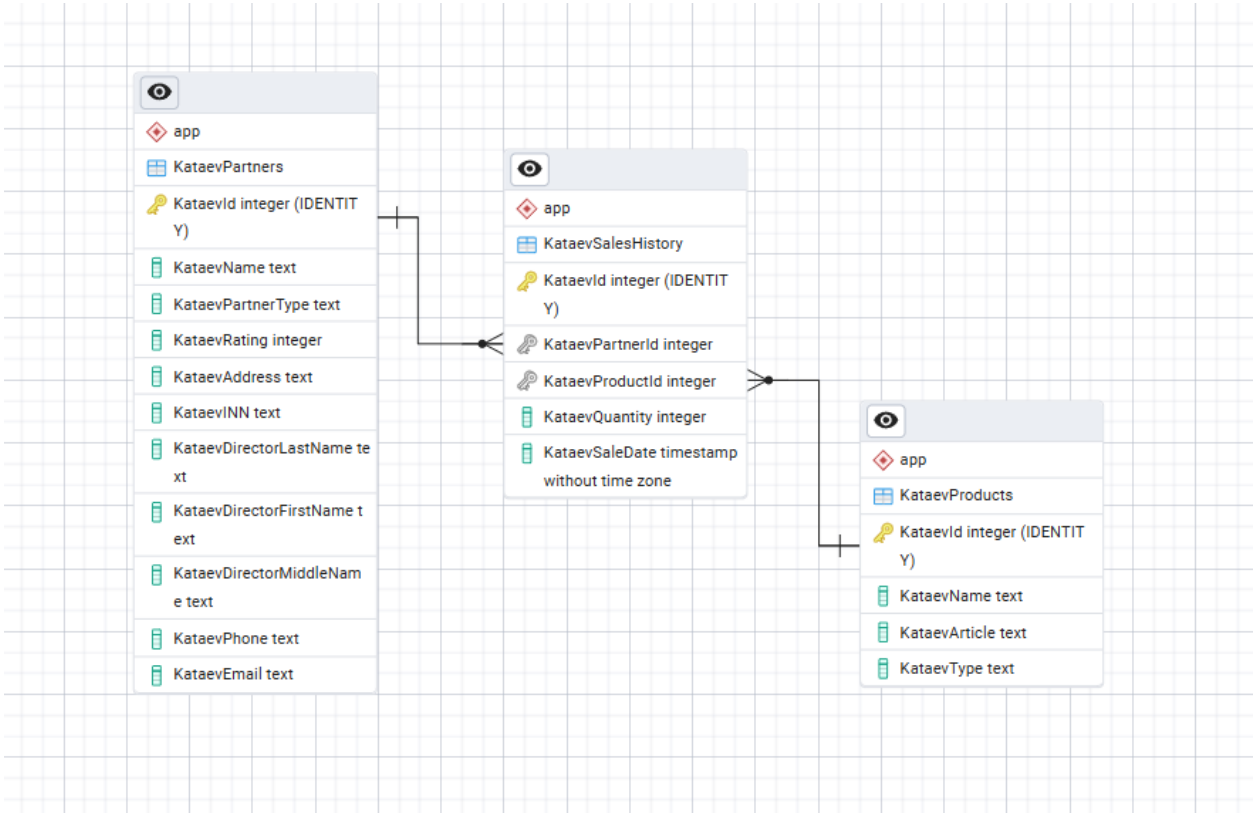

Рисунок 3 ER-Диаграмма

Разработка библиотеки классов

Основная логика приложения реализована в виде отдельной библиотеки классов KataevLIB, что обеспечивает разделение бизнес-логики и пользовательского интерфейса. Библиотека содержит пять файлов: модели данных, контекст Entity Framework Core, интерфейс и класс бизнес-логики.

Модели данных:

Для каждой сущности предметной области создан отдельный POCO-класс с атрибутами маппинга. Атрибут [Table] указывает имя таблицы и схему в базе данных, [Key] обозначает первичный ключ, [Column] задаёт точное имя столбца, [ForeignKey] определяет внешний ключ для навигационных свойств.

Класс KataevPartner отображается на таблицу KataevPartners схемы app и содержит одиннадцать полей: идентификатор, наименование, тип организации, рейтинг, адрес, ИНН, фамилию, имя и отчество директора, телефон и электронную почту. Навигационное свойство KataevSalesHistories реализует связь один-ко-многим с историей продаж.

Класс KataevProduct отображается на таблицу KataevProducts и хранит наименование, артикул и тип продукции. Аналогично содержит навигационное свойство для связи с историей продаж.

Класс KataevSalesHistory отображается на таблицу KataevSalesHistory и связывает партнёра с продукцией через внешние ключи KataevPartnerId и KataevProductId, а также хранит количество и дату продажи. Навигационные свойства KataevPartner и KataevProduct обеспечивают загрузку связанных сущностей.

Контекст базы данных:

Класс ApplicationContext наследуется от DbContext и является точкой входа для всех операций с базой данных через Entity Framework Core. Он содержит три свойства типа DbSet<T>, соответствующие трём таблицам: KataevPartners, KataevProducts и KataevSalesHistories.

В конструкторе по умолчанию вызывается Database.EnsureCreated(), что обеспечивает автоматическое создание таблиц при первом запуске

приложения, если они ещё не существуют. Метод OnConfiguring настраивает подключение к PostgreSQL через провайдера Npgsql, указывая хост, порт, имя базы данных kataev и учётные данные роли app. Дополнительный конструктор, принимающий DbContextOptions, предусмотрен для передачи настроек извне, что используется в тестовом проекте.

Интерфейс IMainApp:

Для обеспечения слабой связанности между библиотекой и WPF-приложением был разработан интерфейс IMainApp, объявляющий контракт всех публичных операций системы. Интерфейс содержит методы управления контекстом (GetContext, SaveChanges, SetupAppSchema), полный набор CRUD-операций для каждой из трёх сущностей, а также метод бизнес-логики CalculatePartnerDiscount. Использование интерфейса позволяет подменять реализацию без изменения кода потребителей, что было использовано при написании модульных тестов.

Класс MainApp:

Класс MainApp реализует интерфейс IMainApp и содержит всю бизнес-логику приложения. Он реализует паттерн IDisposable для корректного освобождения ресурса контекста базы данных: при вызове Dispose контекст уничтожается, флаг disposedValue предотвращает повторное освобождение, а финализатор обеспечивает освобождение ресурсов в случае, если Dispose не был вызван явно.

В конструкторе вызывается метод SetupAppSchema, который при каждом запуске приложения убеждается в существовании схемы app, устанавливает путь поиска и предоставляет роли app необходимые права доступа посредством прямого выполнения SQL-команд через Database.ExecuteSqlRaw.

CRUD-методы для партнёров, продукции и истории продаж делегируют операции соответствующим DbSet контекста. Метод AddSalesHistory дополнительно проверяет существование партнёра с указанным идентификатором и выбрасывает исключение при его отсутствии.

Методы GetPartners и GetPartner используют Include для жадной загрузки связанной истории продаж, а GetSalesHistory — для загрузки связанной продукции.

Ключевым методом бизнес-логики является CalculatePartnerDiscount, вычисляющий индивидуальную скидку партнёра. Метод суммирует количество всей реализованной продукции по идентификатору партнёра и возвращает процент скидки согласно пороговым значениям: менее 10 000 единиц — 0%, от 10 000 до 50 000 — 5%, от 50 000 до 300 000 — 10%, свыше 300 000 — 15%.

Диаграмма классов

Была построена диаграмма классов основной библиотеки KataevLIB. Основные классы:

- ApplicationContext — контекст Entity Framework
- KataevPartner — сущность партнёра (один ко многим с продажами)
- KataevProduct — сущность продукции
- KataevSalesHistory — история продаж (связывает партнёра и продукцию)
- MainApp — бизнес-логика приложения (CRUD-операции + CalculatePartnerDiscount)
- IMainApp — интерфейс для внедрения зависимостей

Связи: один-ко-многим (Partner - SalesHistory, Product - SalesHistory). Все таблицы находятся в схеме app.

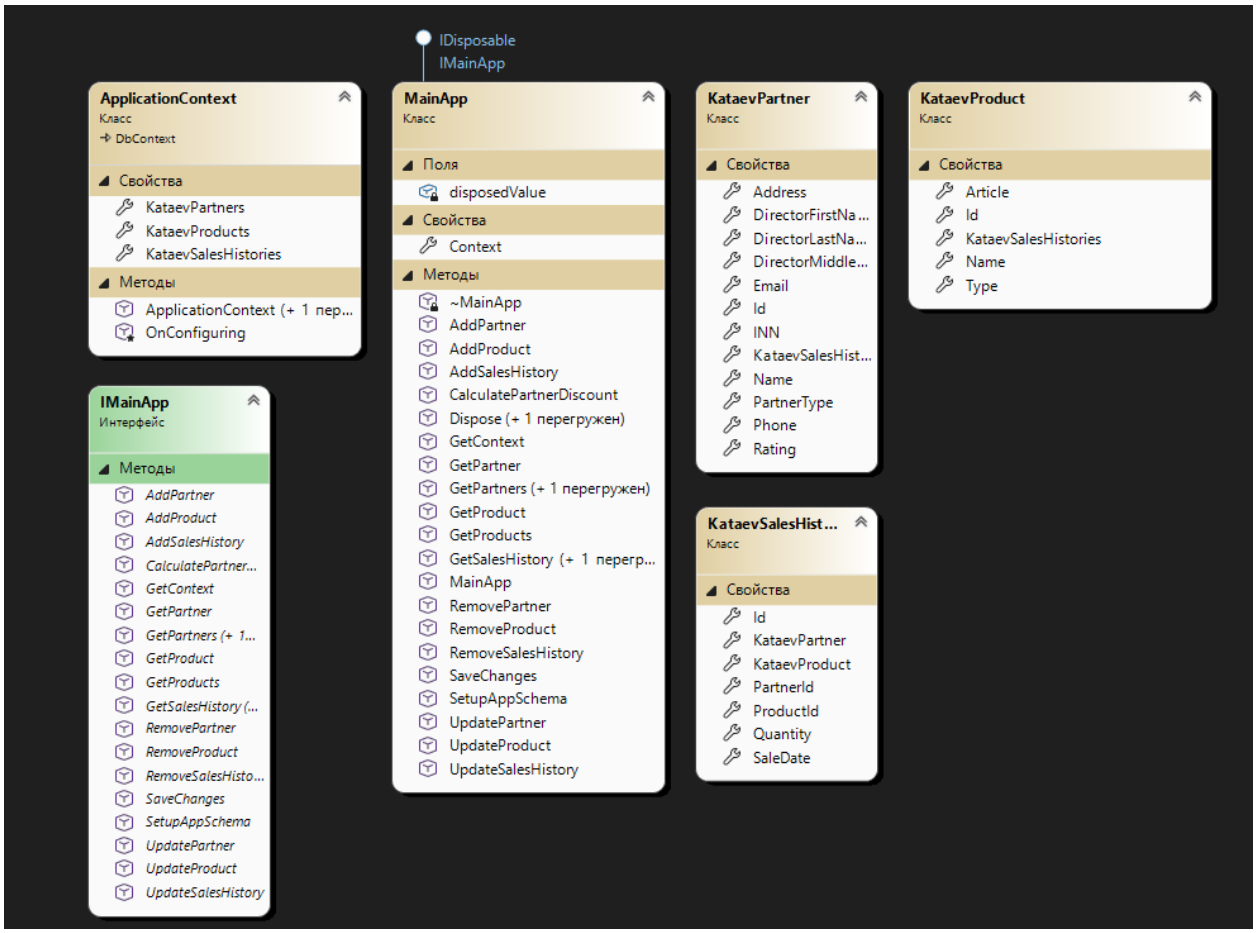

Рисунок 4 Диаграмма классов библиотеки

Разработка форм для ввода, просмотра и редактирования данных

Описание главной формы (MainWindow):

Главная форма приложения реализует мастер-детальный интерфейс для управления партнерами и их заказами. Компоновка выполнена на базе контейнера Grid и включает четыре функциональные зоны:

1. Заголовок и панель инструментов: содержат наименование системы и кнопки управления партнерами (добавление, редактирование, удаление, управление продукцией). Состояние кнопок редактирования и удаления блокируется при отсутствии выбранного элемента.
2. Список партнеров (левая панель): Реализован через элемент ListBox с кастомным шаблоном (DataTemplate). Каждая запись отображается в виде карточки с ключевой информацией (название, тип, рейтинг) и кнопкой быстрого добавления заказа. Выбранный элемент выделяется цветом и акцентной полосой.
3. Панель деталей (правая панель): активируется при выборе партнера. Отображает сводную информацию (контакты, скидка, объем продаж) и модуль управления заказами. Включает таблицу DataGrid со списком транзакций (дата, продукция, артикул, количество), настроенную в режим «только чтение».
4. Строка состояния: выводит итоговое количество партнеров в системе.

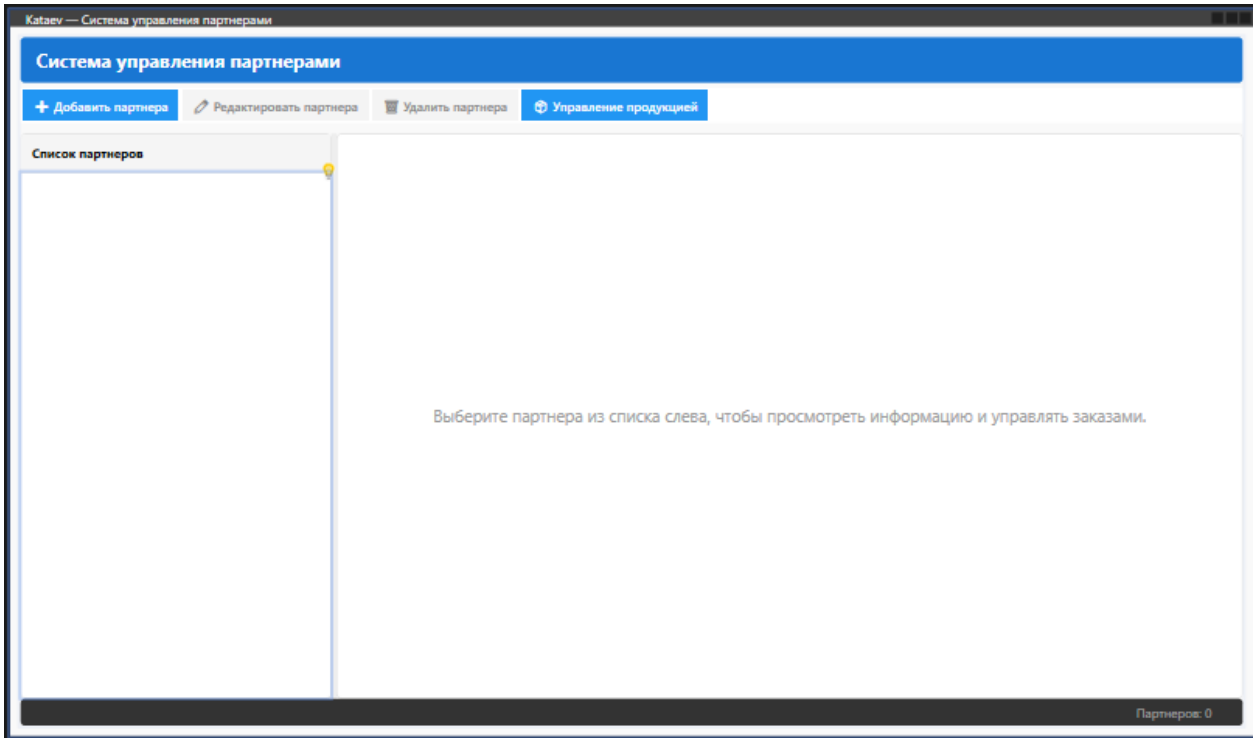

Рисунок 5 Главная форма MainWindow

Описание формы редактирования партнера (KataevPartnerEditWindow):

Форма KataevPartnerEditWindow предназначена для создания новых записей о партнерах и модификации существующих. Окно имеет фиксированный размер, центрируется относительно родительского окна и выполнено в едином стиле с основным интерфейсом.

Структура формы:

1. Область ввода данных: Основное пространство занимает прокручиваемая панель (ScrollViewer), содержащая сетку (Grid) из пар «метка – поле ввода». Это гарантирует доступность всех элементов даже при ограниченной высоте экрана.
2. Состав полей: Форма включает полный набор атрибутов, необходимых для учета партнера:
   - Основные данные: Наименование организации, тип партнерства (выбирается из выпадающего списка ComboBox: оптовый покупатель, розничный магазин и т.д.), рейтинг (числовое поле с ограничением ввода).
   - Юридическая информация: ИНН, юридический адрес.
   - Контактные лица: Фамилия, имя и отчество директора (разделены на три отдельных поля для структурированного хранения).
   - Связь: Телефон и адрес электронной почты.

3. Панель управления: В нижней части расположены кнопки подтверждения действий («Сохранить», «Отмена»), выровненные по правому краю. Кнопки имеют цветовую кодировку (зеленый для подтверждения, красный для отмены) для интуитивного понимания их назначения.

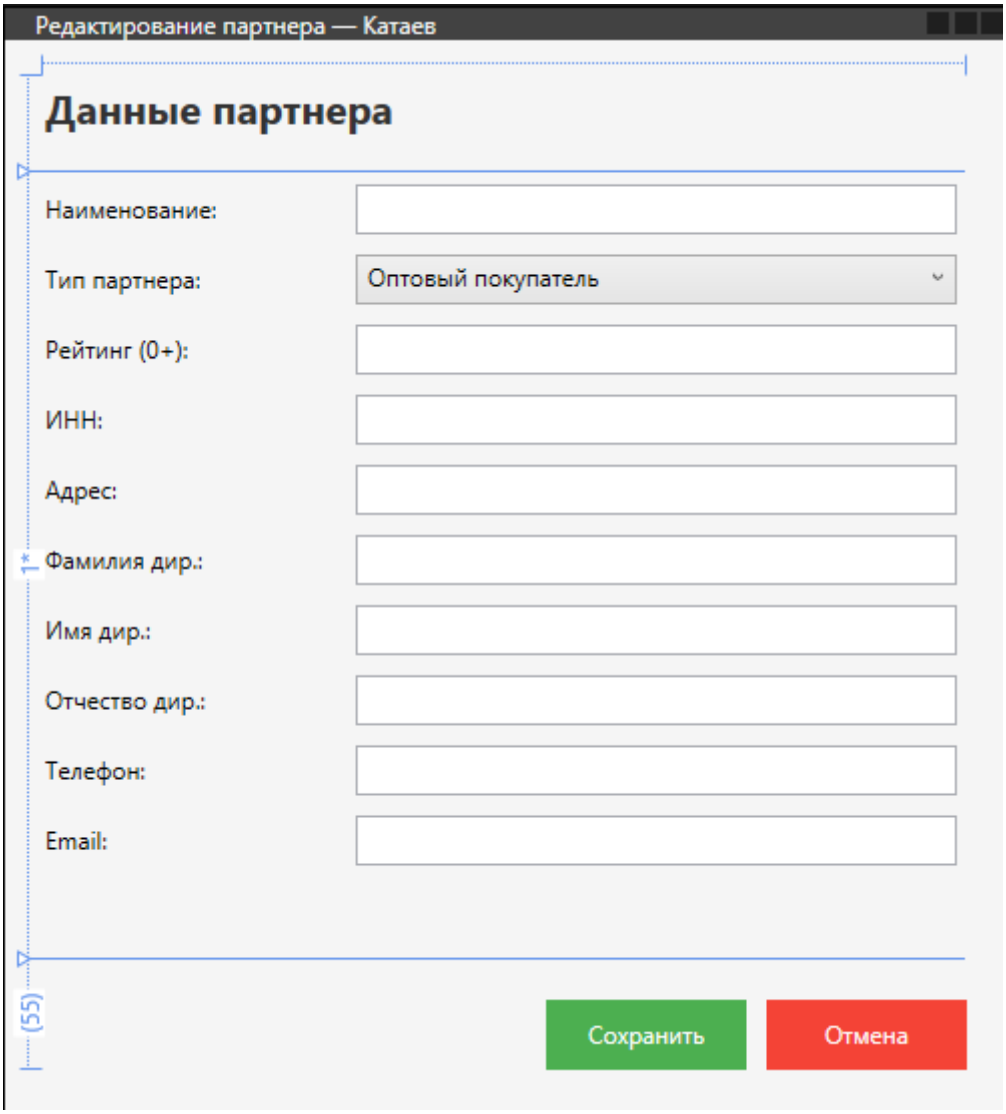

Рисунок 6 форма редактирования партнера

Описание форм управления продукцией:

Для работы с номенклатурой товаров в системе реализован модуль, состоящий из двух взаимосвязанных форм: списка продукции и редактора карточки товара.

1. Форма списка продукции (KataevProductsWindow) Данное окно служит реестром всех доступных товаров.

- Отображение данных: Основную часть интерфейса занимает таблица DataGrid, работающая в режиме «только чтение». Она отображает ключевые атрибуты товара: уникальный идентификатор (ID), наименование, артикул и тип. Колонка наименования автоматически расширяется, занимая все свободное пространство.
- Управление: В нижней части расположена панель инструментов с кнопками для выполнения CRUD-операций: добавление, редактирование и удаление записей. Кнопки модификации и удаления активны только при выделении строки в таблице. Также предусмотрена кнопка закрытия окна.

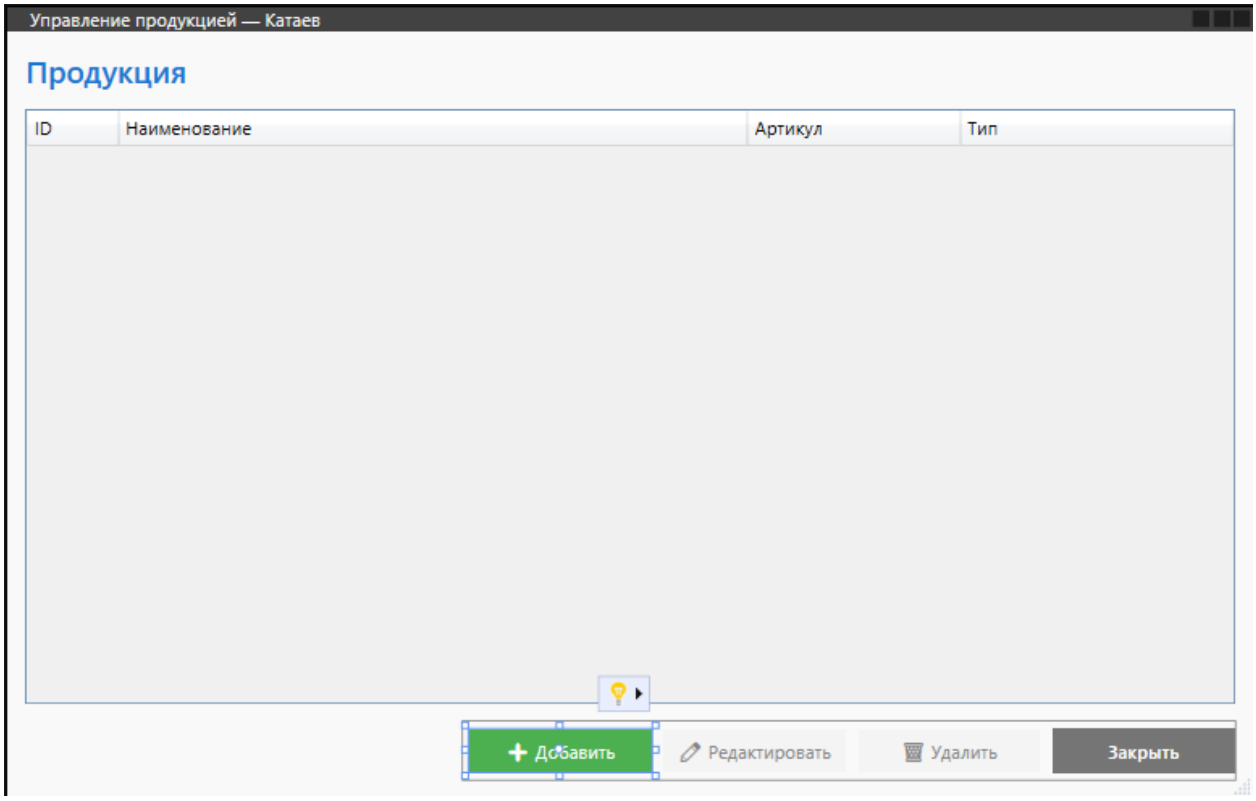

Рисунок 7 форма просмотра добавления продукции

2. Форма редактирования товара (KataevProductEditWindow) Модальное окно, предназначенное для ввода и коррекции данных о конкретном товаре.

- Структура: Интерфейс выполнен в виде компактной формы с фиксированными размерами. Заголовок окна динамически меняется в зависимости от режима работы («Добавление» или «Редактирование»).
- Поля ввода: Предусмотрены три текстовых поля для заполнения обязательных характеристик: наименование, артикул и тип продукции.
- Завершение работы: внизу расположены кнопки «Сохранить» и «Отмена», позволяющие подтвердить внесение изменений или закрыть форму без сохранения.

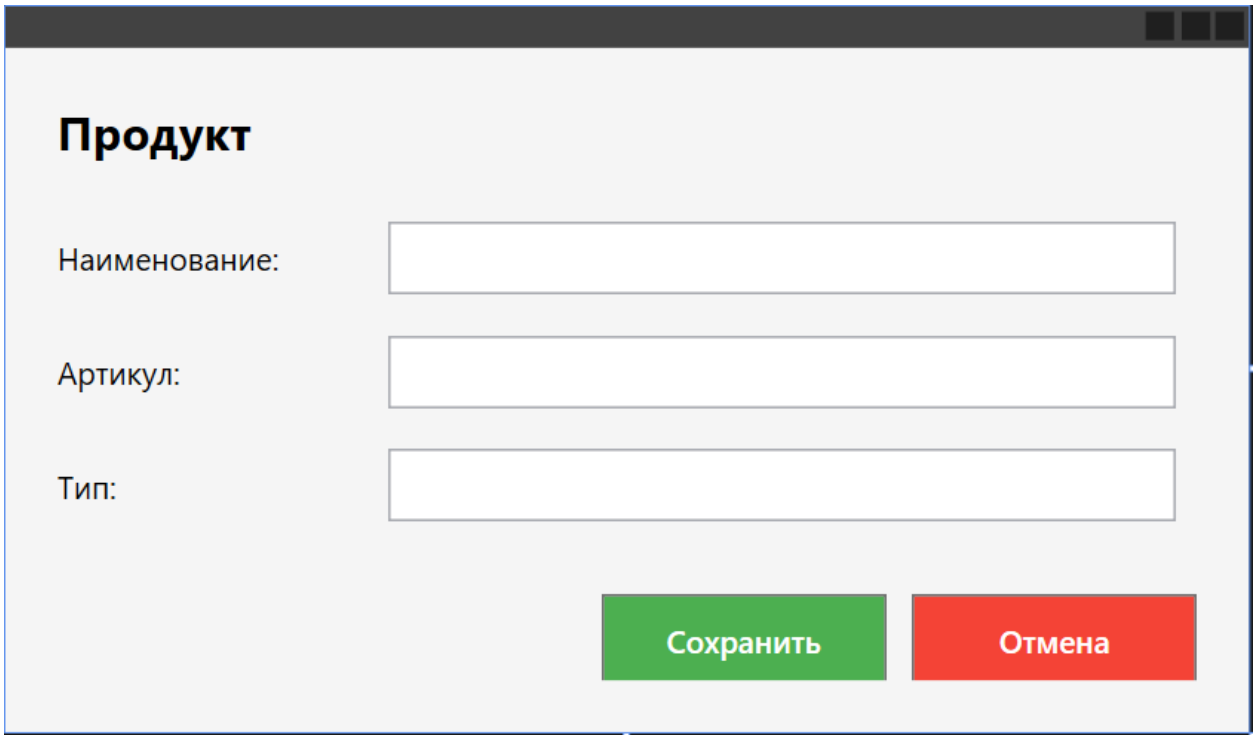

Рисунок 8 форма создания продукции

Описание формы редактирования заказа (KataevOrderEditWindow):

Форма KataevOrderEditWindow представляет собой модальное окно для создания новых записей о продажах или изменения параметров существующих заказов. Интерфейс оптимизирован для быстрого ввода ключевых транзакционных данных.

Основные элементы управления:

1. Выбор продукции: поле типа ComboBox позволяет выбрать товар из доступной номенклатуры. Элемент настроен на отображение наименования товара (DisplayMemberPath="Name") при сохранении внутреннего идентификатора (SelectedValuePath="Id"), что обеспечивает корректную связь с базой данных. Реализована подсказка (placeholder) для облегчения навигации пользователю.
2. Ввод количества: текстовое поле (TextBox) предназначено для указания объема партии. Поле выровнено по правому краю для удобства работы с числами и оснащено фильтром ввода (PreviewTextInput), разрешающим ввод только цифровых символов, что исключает ошибки формата данных.
3. Дата операции: для фиксации даты продажи используется стандартный компонент DatePicker, предоставляющий удобный календарный интерфейс выбора и гарантирующий валидность формата даты.

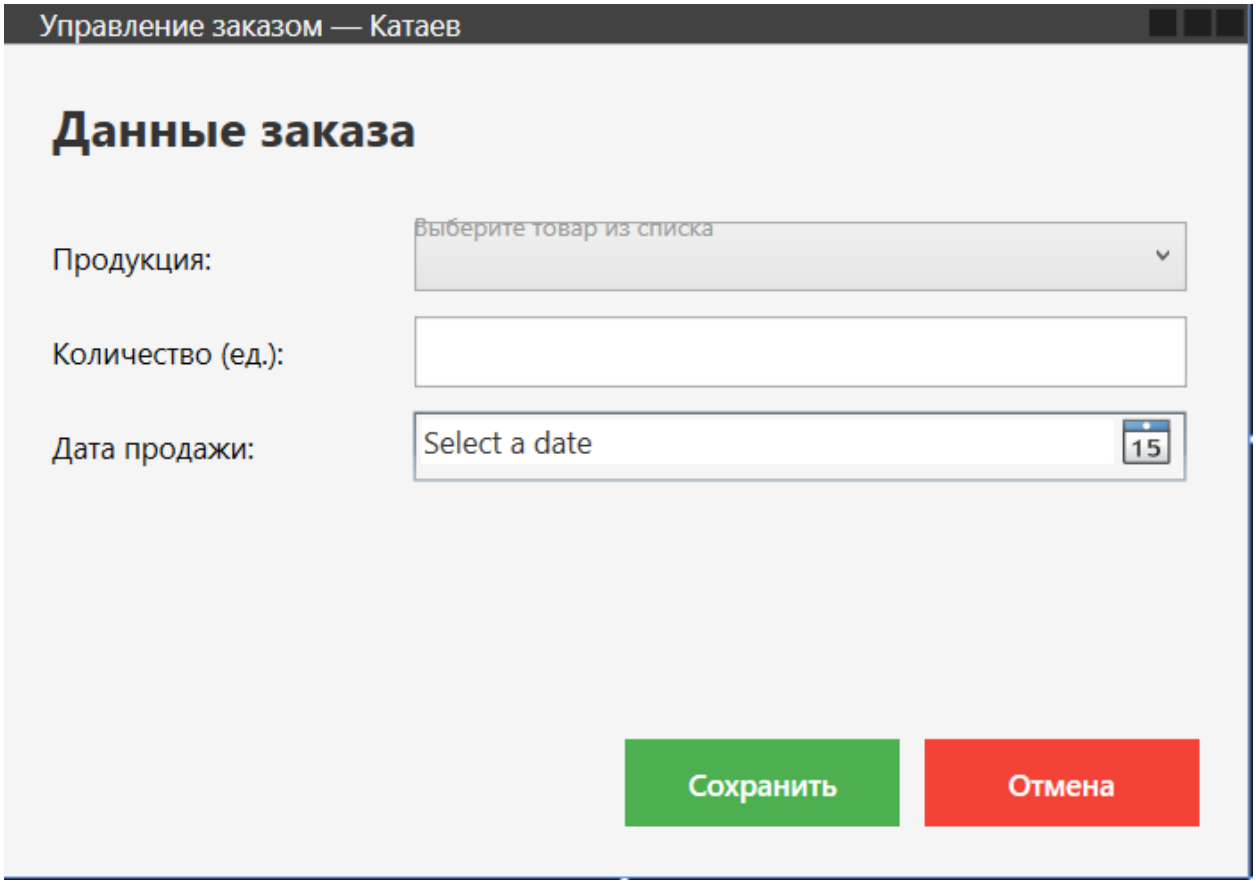

Рисунок 9 форма создания заказа

Этапы разработки интерфейса

1. Проектирование — определение необходимых окон и элементов управления.
2. Прототипирование — создание каркаса окон в Visual Studio XAML Designer, привязка данных через ItemsSource.
3. Стилизация — вынесены глобальные стили (ActionButton, ToolbarButton) в App.xaml, применены единые цвета (#1976D2, #4CAF50 и др.), добавлены визуальные эффекты (тени, акцентные полоски при выборе партнёра), использованы Material Design-подобные элементы (иконки, бейджи скидок).

App.xaml

Application

```
1   <Application x:Class="KataevWPF.App"
2                xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
3                xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
4                StartupUri="MainWindow.xaml">
5       <Application.Resources>
6           <Style x:Key="ToolbarButton" TargetType="Button">
7               <Setter Property="Padding"      Value="10,6"/>
8               <Setter Property="Margin"       Value="2"/>
9               <Setter Property="Background"   Value="#2196F3"/>
10              <Setter Property="Foreground"   Value="White"/>
11              <Setter Property="BorderThickness" Value="0"/>
12              <Setter Property="Cursor"       Value="Hand"/>
13              <Setter Property="FontWeight"   Value="SemiBold"/>
14              <Setter Property="Height"       Value="30"/>
15              <Style.Triggers>
16                  <Trigger Property="IsMouseOver" Value="True">
17                      <Setter Property="Background" Value="#1976D2"/>
18                  </Trigger>
19                  <Trigger Property="IsEnabled" Value="False">
20                      <Setter Property="Background" Value="#BDBDBD"/>
21                  </Trigger>
22              </Style.Triggers>
23          </Style>
24
25          <Style x:Key="ActionButton" TargetType="Button">
26              <Setter Property="Padding"      Value="8,4"/>
27              <Setter Property="Margin"       Value="2"/>
28              <Setter Property="Background"   Value="#4CAF50"/>
29              <Setter Property="Foreground"   Value="White"/>
30              <Setter Property="BorderThickness" Value="0"/>
31              <Setter Property="Cursor"       Value="Hand"/>
32              <Setter Property="Height"       Value="28"/>
33              <Style.Triggers>
34                  <Trigger Property="IsMouseOver" Value="True">
35                      <Setter Property="Background" Value="#388E3C"/>
36                  </Trigger>
37                  <Trigger Property="IsEnabled" Value="False">
38                      <Setter Property="Background" Value="#BDBDBD"/>
39                  </Trigger>
40              </Style.Triggers>
41          </Style>
42
43          <BooleanToVisibilityConverter x:Key="BoolToVis"/>
44
45      </Application.Resources>
46  </Application>
```

Рисунок 10 файл App.xaml

Все формы соответствуют единому стилю приложения, имеют корректные заголовки и обработку исключений. После каждого этапа проводилось тестирование на работоспособность и соответствие требованиям задания.

3.1.3. Использование методов для получения кода с заданной функциональностью и степенью качества

В процессе разработки подсистемы «Учёт партнёров» применялись современные методы и подходы, обеспечивающие получение кода с чётко заданной функциональностью и высоким уровнем качества.

Основные методы получения кода:

1. Code-First подход с Entity Framework Core. Все модели данных (KataevPartner, KataevProduct, KataevSalesHistory) были созданы как POCO-классы с атрибутами [Table], [Key], [Column] и [ForeignKey]. Контекст ApplicationContext автоматически генерировал и обновлял схему базы данных. Это позволило быстро получать полностью рабочий слой доступа к данным без ручного написания SQL-запросов.
2. LINQ-запросы для бизнес-логики Все выборки данных реализованы через LINQ Пример — метод CalculatePartnerDiscount():
3. Паттерн Repository + Service Вся бизнес-логика сосредоточена в классе MainApp, реализующем интерфейс IMainApp. Это позволило отделить логику от представления (WPF-окна) и легко подменять реализацию при необходимости.
4. Partial-классы и разделение ответственности WPF-окна (MainWindow, KataevProductsWindow и др.) разделены на .xaml (разметка) и .xaml.cs (логика), что соответствует принципам MVVM-подобной архитектуры даже в чистом code-behind.
5. Partial-классы и разделение ответственности WPF-окна (MainWindow, KataevProductsWindow и др.) разделены на .xaml (разметка) и .xaml.cs (логика), что соответствует принципам MVVM-подобной архитектуры даже в чистом code-behind.

Методы обеспечения качества кода:

- Единый стиль именования (префикс Kataev для всех таблиц/классов, CamelCase для свойств).
- Обработка всех возможных исключений (try-catch с MessageBox).
- Комментарии в ключевых местах (бизнес-логика расчёта скидки, настройка схемы БД).
- Модульная структура проекта (отдельная библиотека KataevLIB + WPF-приложение).

Таким образом, применяемые методы позволили получить код с предсказуемой функциональностью, высокой поддерживаемостью и минимальным количеством ошибок.

3.1.4. Инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования;

Для контроля качества исходного кода проводилось регулярное инспектирование компонентов программного обеспечения.

Виды инспекции:

1. Статический анализ кода (автоматический).
2. Ручной code review (самопроверка + проверка преподавателем).
3. Peer-review на финальном этапе перед сдачей.

Актуальные стандарты кодирования:

1. Microsoft C# Coding Conventions (официальные рекомендации Microsoft).
2. .NET Framework Design Guidelines.
3. Собственные правила проекта:
   - Использование CamelCase для свойств и методов.
   - Единый стиль отступов и форматирования (настроен в Visual Studio).

Программы и инструменты инспекции:

1. Встроенный анализатор Visual Studio 2022 (Roslyn Analyzer) — автоматически подсвечивал предупреждения CA, IDE и стилистические нарушения.
2. Manual review — просмотр всего кода перед каждым коммитом и перед финальной сдачей.

В результате инспекции были устранены все предупреждения уровня Error и Warning, код приведён к единому стилю, а нарушения стандартов отсутствуют.

3.1.5. Выполнение отладки программного модуля с использованием специализированных программных средств;

Инспектирование компонент программного обеспечения представляет собой систематический процесс анализа исходного кода с целью выявления отклонений от принятых стандартов, потенциальных дефектов и нарушений архитектурных решений. В рамках производственной практики контроль качества исходного кода осуществлялся на протяжении всего цикла разработки, а не только на финальном этапе, что позволило своевременно выявлять и устранять нарушения.

Процесс проведения отладки:

1. Точечная отладка (Breakpoints) Устанавливались точки останова в ключевых методах:
   - CalculatePartnerDiscount() — проверка правильности расчёта скидки при разных объёмах.
   - AddSalesHistory() — проверка проверки существования партнёра.
   - LoadPartnerDetails() — отладка обновления интерфейса после изменения заказов.
2. Окна отладки
   - Locals и Autos — просмотр значений переменных (totalQuantity, discount, partner.Id).
   - Watch — отслеживание выражений app.GetSalesHistory(partner.Id).Sum(...).
   - Call Stack — анализ цепочки вызовов при возникновении исключений.
   - Immediate Window — оперативное выполнение LINQ-запросов во время отладки.
3. Отладка исключений Включён режим «Break When Thrown» для Exception и DbUpdateException. Все исключения (например, отсутствие партнёра при добавлении продажи) перехватывались и отображались пользователю в понятном виде.

4. Отладка WPF-интерфейса
   - Отладка привязок данных (DataContext, ItemsSource).
   - Использование Live Visual Tree и Live Property Explorer для анализа XAML-элементов.
   - Проверка обновления интерфейса после вызова LoadPartnerDetails() и LoadOrders().
5. Отладка взаимодействия с БД Включено логирование SQL-запросов через optionsBuilder.LogTo(Console.WriteLine) в OnConfiguring, что позволило видеть все генерируемые Entity Framework запросы.
6. Модульное тестирование Для метода CalculatePartnerDiscount() написаны unit-тесты (MSTest), проверяющие все пороги скидки (0%, 5%, 10%, 15%).

В результате отладки все выявленные ошибки (неправильное обновление скидки, проблемы с навигационными свойствами EF Core, исключения при удалении) были устранены. Программа работает стабильно, без аварийных завершений и соответствует требованиям задания.

3.1.6. Осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения

Тестирование программного обеспечения является неотъемлемым этапом разработки, обеспечивающим соответствие реализованных функций предъявляемым требованиям. В рамках производственной практики был разработан комплекс тестовых наборов и тестовых сценариев для подсистемы «Учёт партнёров и расчёт индивидуальных скидок».

Процесс проведения тестирования программного обеспечения

Тестирование проводилось в два этапа: модульное (unit) тестирование и ручное функциональное тестирование.

На этапе планирования были определены объекты тестирования: методы интерфейса IMainApp, реализованные в классе MainApp. Для каждого метода сформулированы тестовые сценарии, покрывающие как штатные, так и граничные случаи исполнения.

Для автоматизированного модульного тестирования использовался фреймворк MSTest (Microsoft.VisualStudio.TestTools.UnitTesting), входящий в состав Visual Studio 2022. Тестовый проект KataevLIB.Tests подключался к основной библиотеке KataevLIB в качестве зависимости.

Поскольку тестируемые методы реализуют интерфейс IMainApp, для изоляции тестов от внешних зависимостей (базы данных PostgreSQL) был применён подход Mock-объектов: реализован класс MockMainApp, имплементирующий IMainApp посредством внутренних коллекций List<T>. Данный подход обеспечил полную независимость тестов от состояния базы данных, воспроизводимость результатов и высокую скорость выполнения.

Разработанные тестовые сценарии

Тестовые сценарии были сгруппированы по функциональным областям:

1. *Управление партнёрами:* проверка добавления партнёра с корректными данными; проверка добавления нескольких партнёров; проверка присвоения уникальных идентификаторов; получение списка при отсутствии записей; поиск по наименованию с совпадением и без; получение партнёра по существующему и несуществующему идентификатору; обновление полей партнёра; удаление партнёра без влияния на остальные записи.
2. *Управление продукцией:* добавление продукта с проверкой всех полей; получение списка при отсутствии и наличии записей; получение по идентификатору; обновление типа и артикула; удаление продукта.
3. *История продаж:* добавление записи при существующем партнёре; генерация исключения при несуществующем PartnerId; фильтрация записей по идентификатору партнёра; получение истории через объект партнёра и через идентификатор; обновление количества; удаление записи.
4. *Расчёт скидки:* проверка всех пороговых значений согласно бизнес-логике (0%, 5%, 10%, 15%); проверка суммирования количества по нескольким записям; проверка нулевого результата при отсутствии продаж.

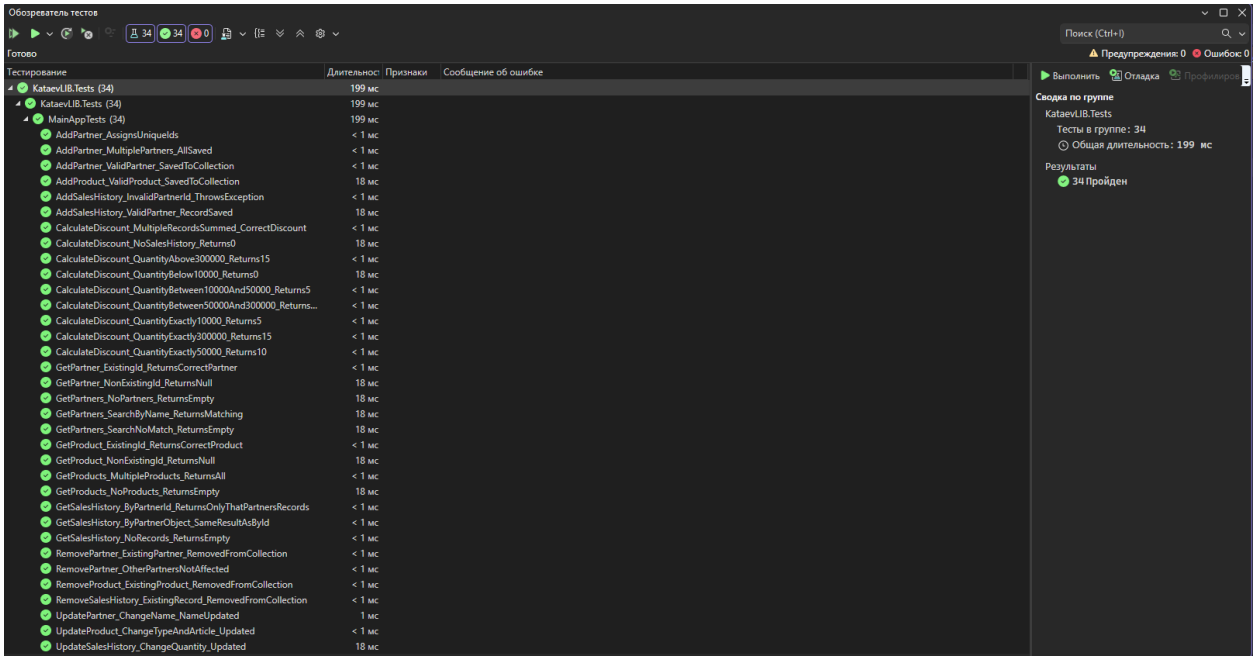

Рисунок 11 результаты прохождения тестирования

Итого разработано 34 тестовых сценария. Все тесты успешно пройдены, что подтверждает корректность реализованной бизнес-логики.

3.1.7. Интегрирование программных модулей

Интеграция программных модулей представляет собой процесс объединения отдельно разработанных компонентов системы в единое функционирующее приложение. В рамках данного проекта применялась пошаговая (инкрементальная) стратегия интеграции, при которой модули подключались последовательно по мере их готовности.

Архитектурно проект состоит из трёх модулей:

- KataevLIB — библиотека классов, содержащая модели данных, контекст Entity Framework Core и бизнес-логику (ApplicationContext, KataevPartner, KataevProduct, KataevSalesHistory, MainApp, IMainApp);
- KataevWPF — графический интерфейс пользователя на базе Windows Presentation Foundation, обеспечивающий взаимодействие с конечным пользователем;
- KataevLIB.Tests — проект модульных тестов, верифицирующий корректность бизнес-логики.

Процесс интеграции осуществлялся в следующей последовательности:

1. На первом этапе была разработана и протестирована библиотека KataevLIB как самостоятельный модуль. Модели данных и контекст подключения к PostgreSQL проверялись посредством прямого обращения к базе данных через Entity Framework Core.
2. На втором этапе выполнена интеграция библиотеки KataevLIB с WPF-приложением. Проект KataevWPF подключил KataevLIB в качестве ссылки на проект (Project Reference). Экземпляр класса MainApp создавался в конструкторе главного окна MainWindow и передавался в дочерние окна через параметры конструктора, что реализует принцип внедрения зависимостей (Dependency Injection) через конструктор.
3. На третьем этапе тестовый проект KataevLIB.Tests был интегрирован с KataevLIB посредством аналогичной ссылки на проект. Поскольку тесты используют MockMainApp, реализующий интерфейс IMainApp,

зависимость от конкретной реализации MainApp и от PostgreSQL полностью отсутствует.

Управление зависимостями и версиями пакетов осуществлялось через NuGet Package Manager. Ключевые пакеты: EntityFrameworkCore 8.x, Npgsql.EntityFrameworkCore.PostgreSQL, Все модули целевой платформой использовали .NET Framework, что обеспечило совместимость между проектами.

Контроль корректности интеграции обеспечивался запуском автоматических тестов после каждого значимого изменения, а также ручной проверкой сценариев использования в работающем WPF-приложении.

3.1.8. Верификация и аттестация программного обеспечения

Верификация и аттестация являются завершающими процессами обеспечения качества программного обеспечения, подтверждающими соответственно корректность реализации и соответствие продукта потребностям заказчика.

Верификация отвечает на вопрос «правильно ли построена система?» и направлена на проверку соответствия программного обеспечения технической спецификации и проектным решениям.

В рамках верификации были выполнены следующие мероприятия:

1. *Статический анализ* — инспектирование исходного кода с использованием встроенного анализатора Roslyn в Visual Studio 2022. Проверялось соответствие соглашениям именования, отсутствие неиспользуемых переменных, корректность обработки исключений и полнота комментариев.
2. *Модульное тестирование* — выполнение 34 разработанных unit-тестов, покрывающих все методы интерфейса IMainApp. Тесты подтвердили корректность реализации CRUD-операций и алгоритма расчёта скидки во всех граничных случаях.
3. *Проверка схемы базы данных* — верификация соответствия созданных таблиц проектной документации: наличие схемы app, корректность префиксов Kataev, правильность внешних ключей и ограничений.

Аттестация отвечает на вопрос «правильная ли система построена?» и направлена на подтверждение соответствия программного обеспечения требованиям заказчика, изложенным в техническом задании (Приложение 1).

В рамках аттестации проводилось функциональное тестирование методом чёрного ящика — проверка работоспособности приложения с позиции конечного пользователя без учёта внутренней реализации. Тестировались следующие пользовательские сценарии:

1. просмотр списка партнёров с отображением текущей скидки;
2. добавление, редактирование и удаление партнёра через формы ввода;
3. ведение истории продаж: добавление и удаление записей;
4. автоматический пересчёт скидки после изменения объёма продаж;
5. управление справочником продукции;
6. корректное отображение сообщений об ошибках при некорректном вводе.

Все перечисленные сценарии подтвердили соответствие разработанного программного обеспечения требованиям технического задания. Приложение функционирует стабильно, без аварийных завершений. Расчёт скидки соответствует заданным пороговым значениям. Интерфейс обеспечивает выполнение всех предусмотренных операций.

По результатам верификации и аттестации программный продукт признан готовым к эксплуатации.

ЗАКЛЮЧЕНИЕ

В результате прохождения производственной практики с 09 по 16 марта 2026г. были закреплены теоретические знания и расширены профессиональные умения в области разработки, интеграции и тестирования программного обеспечения.

По итогам практики получены следующие результаты:

1. Разработана подсистема «Учёт партнёров и расчёт индивидуальных скидок» для производственной компании «Мастер пол», включающая библиотеку классов KataevLIB, WPF-приложение и проект модульных тестов.
2. Спроектирована и реализована реляционная база данных в СУБД PostgreSQL 18 в схеме app, содержащая три взаимосвязанные таблицы (партнёры, продукция, история продаж), приведённые к третьей нормальной форме.
3. Реализована бизнес-логика автоматического расчёта индивидуальной скидки партнёра на основании суммарного объёма реализованной продукции с четырьмя пороговыми значениями: 0%, 5%, 10% и 15%.
4. Разработан пользовательский интерфейс на базе технологии WPF, обеспечивающий выполнение полного цикла операций по работе с данными: добавление, редактирование, удаление и извлечение записей для всех сущностей предметной области.
5. Выполнена интеграция программных модулей: библиотека классов KataevLIB подключена к WPF-приложению и тестовому проекту посредством ссылок на проект с применением принципа внедрения зависимостей через интерфейс IMainApp.
6. Разработан комплекс из 34 модульных тестов на платформе MSTest, охватывающих все методы интерфейса IMainApp, включая граничные случаи расчёта скидки и проверку корректности обработки исключений.
7. Организован контроль версий исходного кода с использованием системы Git и облачного хостинга GitHub; исходный код проекта и

сопроводительная записка размещены в репозитории в открытом доступе.

По окончании практики был получен практический опыт интеграции модулей в программное обеспечение; отладки программного модуля с использованием специализированных программных средств Visual Studio 2022; разработки тестовых наборов и тестовых сценариев для программного обеспечения с применением методологии Mock-объектов; инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

Основные источники

1 Стружкин, Н. П. Базы данных: проектирование. Практикум : учебное пособие для среднего профессионального образования / Н. П. Стружкин, В. В. Годин. Москва : Издательство Юрайт, 2019. — 291 с. — (Профессиональное образование). — ISBN 978-5-534-08140-4. — Текст : электронный // ЭБС Юрайт [сайт]. — URL: https://biblio-online.ru/bcode/442343

2 Илюшечкин, В. М. Основы использования и проектирования баз данных : учебник для среднего профессионального образования / В. М. Илюшечкин. испр. и доп. — Москва : Издательство Юрайт, 2019. — 213 с. — (Профессиональное образование). — ISBN 978-5-534-01283-5. — Текст : электронный // ЭБС Юрайт [сайт]. — URL: https://biblio-online.ru/bcode/437670

Дополнительные источники

3 Советов, Б. Я. Базы данных : учебник для среднего профессионального образования / Б. Я. Советов, В. В. Цехановский, В. Д. Чертовской. 3-е изд., перераб. и доп. — Москва : Издательство Юрайт, 2019. — 420 с. — (Профессиональное образование). — ISBN 978-5-534-09324-7. — Текст : электронный // ЭБС Юрайт [сайт]. — URL: https://biblio-online.ru/bcode/438438

4 Нестеров, С. А. Базы данных : учебник и практикум для среднего профессионального образования / С. А. Нестеров. Москва : Издательство Юрайт, 2020. — 230 с. — (Профессиональное образование). — ISBN 978-5-534-11629-8. — Текст : электронный // ЭБС Юрайт [сайт]. — URL: https://biblio-online.ru/bcode/457142

5 Стасышин, В. М. Базы данных: технологии доступа : учебное пособие для среднего профессионального образования / В. М. Стасышин, Т. Л. Стасышина 2-е изд., испр. и доп. — Москва : Издательство Юрайт, 2019. — 164 с. — (Профессиональное образование). — ISBN 978-5-534-09888-4. — Текст : электронный // ЭБС Юрайт [сайт]. — URL: https://biblio-online.ru/bcode/442342

6 Голицына, О. Л. Основы проектирования баз данных : учеб. пособие / О.Л. Голицына, Т.Л. Партыка, И.И. Попов. 2-е изд., перераб. и доп. — Москва : ФОРУМ : ИНФРА-М, 2019. — 416 с. : ил. — (Среднее профессиональное образование). - ISBN 978-5-16- 105762-9. - Текст : электронный. - URL: https://znanium.com/catalog/product/1018906

7 Стружкин, Н. П. Базы данных: проектирование : учебник для среднего профессионального образования / Н. П. Стружкин, В. В. Годин. Москва : Издательство Юрайт, 2020. — 477 с. — (Профессиональное образование). — ISBN 978-5-534-11635-9. — Текст : электронный // ЭБС Юрайт [сайт]. — URL: https://biblio-online.ru/bcode/457135

8 Партыка, Т. Л. Информационная безопасность : учеб. пособие / Т.Л. Партыка, И.И. Попов5-е изд., перераб. и доп. — Москва : ФОРУМ : ИНФРА-М, 2018. — 432 с. — (Среднее профессиональное образование). - ISBN 978-5-16-101302-1. - Текст : электронный. - URL: https://znanium.com/catalog/product/915902

9 Грекул , В. И. Проектирование информационных систем : учебник и практикум для среднего профессионального образования / В. И. Грекул, Н. Л. Коровкина, Г. А. Левочкина. Москва : Издательство Юрайт, 2020. — 385 с. — (Профессиональное образование). — ISBN 978-5-534-12104-9. — Текст : электронный // ЭБС Юрайт [сайт]. — URL: https://biblio-online.ru/bcode/457223

III Профессиональные базы данных, информационно-справочные системы

Научная электронная библиотека www.elibrary.ru

Единое окно доступа к образовательным ресурсам. Раздел Информатика и информационные технологии http://window.edu.ru/catalog/?p_rubr=2.2.75.6 on-line библиотека свободно доступных материалов по информационным технологиям на русском языке http://citforum.ru